Register Machines

Tape-based automata, like the Turing Machine, are not the only formal models of computation studied in computer science.

CONTENTS

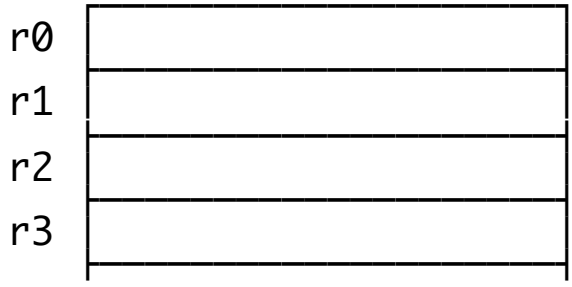
Why Registers? • The Basics • Counter Machines • RAMs • RASPs • Summary

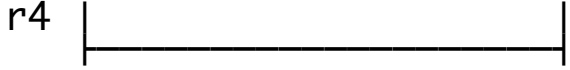
Why Registers?

Tape-based automata are quite minimalistic, and great for making a theory from just about the most primitive concepts. Registers feel a little higher-level, and also, let's face it, real machines have registers.

The Basics

In automata theory, A **register machine** has a number of registers $r_0, r_1, r_2, \ldots$, each holding a single (unbounded) natural number. The machine's control is a program, defined as a list of (labeled) instructions. The instructions are normally executed in sequence, but some instructions can cause a "jump" to an arbitrary instruction. The instruction set varies from model to model (of which there exist an infinite number of possibilities).





Don't miss the Wikipedia article on register machines.

The simplest kind of register machines are those whose instruction set only allows registers to be incremented or decremented only. These are called **counter machines**.

Counter Machines

Here's a sample instruction set (I just made it up) for a counter machine:

Various Counter Machine Instructions	
CLEAR r_i	$r_i := 0$
INC r_i	$r_i := [r_i] + 1$
DEC r_i	$r_i := [r_i] - 1$
COPY r_1, r_j	$r_j := [r_i]$
JZ r_i, L	If $[r_i] = 0$ then jump to L
JE r_i, r_j, L	If $[r_i] = [r_j]$ then jump to L
JZDEC r_i, L	If $[r_i] = 0$ then jump to L else $r_i := [r_i] - 1$
JUMP L	Jump to instruction at address L

This instruction set is pretty simple; it assumes the input is the contents of the registers before execution, and the output is the contents of r_0. The program is a list of instructions and the program ends when you have no more instructions.

For convenience, we can also imagine a machine that has separate streams for input and output, operated on by READ and WRITE instructions, respectively. We might even throw in a convenient instruction to halt the program:

More Instructions	
READ r_i	Consume next value from input stream and store in r_i
WRITE r_i	Append contents of r_i to output stream
HALT	Stop executing

Because counter machine programs are primitive, like Turing Machines, programs are pretty long. Here's a program that reads to integers, adds them, and writes their sum.

```
0 |  READ r0           // x
1 |  READ r1           // y
2 |  JZDEC r1, 5       // if y is 0, we're done
3 |  INC r0            // x++
4 |  JUMP 2            // loop back
5 |  WRITE r0          // there's the output
```

If you have the right set of instructions, you can show a register machine to be equivalent in computing power to a Turing machine.

Surprisingly, there exists a register machine that is equivalent in computing power to a Turing Machine that has only two instructions:

Simplest Counter Machine	
INC r_i	Increment register r_i
JZDEC r_i, L	If register r_i holds 0, jump to instruction at address L, otherwise decrement register r

> ***O RLY?***
>
> *Remember the reads and writes aren't actually needed. You can still compute by having a convention such that the input of the program starts in register 0 and the output is read from register 1 whenever there are no more instructions to execute. Since each register is an unbounded integer, it can encode whatever the heck you like.*

RAMs

A more sophisticated type of register machine is the **RAM** (Random Access Machine). Instruction sets vary, but usually contain arithmetic, logic, and jumps. Some instruction sets feature three-operands (sometimes called 3AC sets) and might look like this:

Restricted 3AC: Simple Direct Addressing Only

Instruction	Description
LOAD n, r_k	$r_k := n$
COPY r_1, r_j	$r_j := [r_i]$
ADD r_i, r_j, r_k	$r_k := [r_i] + [r_j]$
SUB r_i, r_j, r_k	$r_k := [r_i] - [r_j]$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG r_i, r_j	$r_j := -[r_i]$
NOT r_i, r_j	$r_j := \neg[r_i]$
Can also include JUMP, JZ, JE, READ, WRITE, HALT from above	

Now we can improve the model a great deal by adding two powerful ideas, (1) we can let our *source* operands be constant values (also known as immediate values), and (2) let our registers be **indirect**. For example:

```
COPY 5, r8      -- with imm operands, no need for LOAD
COPY 13, r5
COPY r8, r2     -- copy contents of r8 into r2
COPY *r8, r2    -- copy contents of r5 into r2
COPY r8, *r2    -- copy contents of r8 into r13
COPY *r8, *r2   -- copy contents of r5 into r13
NEG *r2, r3     -- copy negation of r13 into r3
ADD *r8, 5, r3
SUB r13, *r13, *r2
```

Now sources can be immediates, direct registers, or indirect registers; destinations can be direct or indirect registers (not immediates!). So we can write:

3AC	
Instruction	**Description**
COPY $src, dest$	$\langle dest \rangle := \langle src \rangle$
ADD $src_1, src_2, dest$	$\langle dest \rangle := \langle src_1 \rangle + \langle src_2 \rangle$
SUB $src_1, src_2, dest$	$\langle dest \rangle := \langle src_1 \rangle - \langle src_2 \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG $src, dest$	$\langle dest \rangle := -\langle src \rangle$

NOT $src, dest$	$\langle dest \rangle := \neg \langle src \rangle$
JZ src, L	If $\langle src \rangle = 0$ then jump to L
JE src_1, src_2, L	If $\langle src_1 \rangle = \langle src_2 \rangle$ then jump to L
JUMP L	Jump to instruction at address L
READ $dest$	$\langle dest \rangle :=$ next value from input stream
WRITE src	Append $\langle src \rangle$ to output stream
HALT	Stop executing

Here is a GCD program on this machine:

```
// while y != 0:
//   x, y = y, x % y
// print(x)

0 |  READ r0            // x
1 |  READ r1            // y
2 |  JZ r1, 7           // if y is 0, we're done
3 |  COPY r1, r2        // y -> old_y (to "save" it)
4 |  MOD r0, r2, r1     // x % y -> y
5 |  COPY r2, r0        // old_y -> x
6 |  JUMP 2             // back to top of "loop"
7 |  WRITE r0
```

Rather than worrying about the position of instructions, let's use **labels**:

```
// while y != 0:
//   x, y = y, x % y
// print(x)

  READ r0            // x will be in r0
  READ r1            // y will be in r1
start:
  JZ r1, wrapup      // if y is 0, we're done
  COPY r1, r2        // y -> old_y
  MOD r0, r2, r1     // x % old_y -> y
  COPY r2, r0        // old_y -> x
  JUMP start         // back to top of "loop"
wrapup:
  WRITE r0           // print x
```

Another variation requires the target to be one of the sources:

2AC	
Instruction	**Description**
COPY $src, dest$	$\langle dest \rangle := \langle src \rangle$
ADD $src, dest$	$\langle dest \rangle := \langle dest \rangle + \langle src \rangle$
SUB $src, dest$	$\langle dest \rangle := \langle dest \rangle - \langle src \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG $dest$	$\langle dest \rangle := -\langle dest \rangle$
NOT $dest$	$\langle dest \rangle := \neg\langle dest \rangle$
JZ src, L	If $\langle src \rangle = 0$ then jump to L
JUMP L	Jump to instruction at address L
READ $dest$	$\langle dest \rangle :=$ next value from input stream
WRITE src	Append $\langle src \rangle$ to output stream
HALT	Stop executing

Code gets a little longer now:

```
// 2AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

  READ r0            // x will be in r0
  READ r1            // y will be in r1
start:
  JZ r1, wrapup      // if y is 0, we're done
  COPY r0, r2        // x -> old_x
  COPY r1, r3        // y -> old_y
  MOD r3, r2         // old_x % old_y
  COPY r2, r1        // new y for next iteration
  COPY r3, r0        // new x for next iteration
  JUMP start         // back to top of "loop"
wrapup:
  WRITE r0           // print x
```

We can even make the target register always be implicitly r_0, which, since this is always the destination for all the arithmetic operations, we call the **accumulator**:

1AC (Accumulator Machine)	
Instruction	**Description**
LOAD src	$r_0 := \langle src \rangle$
STORE $dest$	$\langle dest \rangle := [r_0]$
ADD src	$r_0 := [r_0] + \langle src \rangle$
SUB src	$r_0 := [r_0] - \langle src \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG	$r_0 := -[r_0]$
NOT	$r_0 := \neg[r_0]$
JZ L	If $[r_0] = 0$ then jump to L
JUMP L	Jump to instruction at address L
READ	$r_0 :=$ next value from input stream
WRITE	Append $[r_0]$ to output stream
HALT	Stop executing

Programs start to get a lot longer with this architecture:

```
// 1AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

  READ            // x in r0
  STORE r1        // x in r1
  READ            // y in r0
start:
  JZ wrapup       // if y is 0, we're done
  STORE r2        // old_y saved in r2
  LOAD r1         // x in r0
  STORE r3        // old_x saved in r3
  MOD r2          // x % y in r0
  STORE r1        // new x for next iteration
  LOAD r2         // new y for next iteration
```

```
  JUMP start      // back to top of "loop"
wrapup:
  LOAD r1         // Now x is in r0
  WRITE           // So we can write x
```

We can make all the arguments implicit if we just organize all our registers like a stack:

0AC (Stack Machine)	
Instruction	**Description**
LOAD n	Push n
ADD	Pop into y; pop into x; push $x + y$
SUB	Pop into y; pop into x; push $x - y$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG	Pop into x; push $-x$
NOT	Pop into x; push $\neg x$
JZ L	Pop into x; If $x = 0$ jump to instruction at address L
JUMP L	Jump to instruction at address L
HALT	Stop executing

Stack machines are cool when executing arithmetic expressions. Work left to right. Example:

```
// Stack machine code for 7 - 3 * 8 ** 1 / 3

LOAD 7    // 7
LOAD 3    // 7 3
LOAD 8    // 7 3 8
LOAD 1    // 7 3 8 1
POW       // 7 3 (8**1)
MUL       // 7 (3*(8**1))
LOAD 3    // 7 (3*(8**1)) 3
DIV       // 7 ((3*(8**1))/3)
SUB       // 7-((3*(8**1))/3)
```

To be able to move things around with a stack architecture, we can add DUP and SWAP instructions, or even have a separate stash of labeled registers for storage.

```
// 0AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

// Assuming separate stash of labeled registers for storage,
// BUT all operations can only occur on stack top.
// STORE instruction does a pop.

  READ
  STORE x
  READ
  STORE y
start:
  LOAD y
  JZ wrapup       // if y is 0, we're done
  LOAD x
  LOAD y
  STORE x         // new x is the old y
  LOAD y
  MOD             // x % y
  STORE y         // new y is x % y
  JUMP start      // back to top of "loop"
wrapup:
  LOAD x
  WRITE
```

RASPs

Another kind is called the **RASP** (Random Access Stored Program). Read the Wikipedia article for this one.

> ***Exercise**: What is the historical importance of the RASP model? Why was it created?*

Summary

We've covered:

- ✅ What register machines are and why they are interesting
- ✅ Counter machines
- ✅ The simplest counter machine that is Turing equivalent
- ✅ 3AC RAMs
- ✅ 2AC RAMs
- ✅ 1AC RAMs (accumulator machines)
- ✅ 0AC RAMs (stack machines)
- ✅ RASPs

Computability Theory

Having theories about what computations are and how they are expressed is cool, but how about some insights into what can and cannot be computed?

CONTENTS

History • Models of Computation • Limits of Computation? • A First Non-Decidable Problem • Recognizability • Beyond Recognizable • Reductions • Rice's Theorem • The Chomsky Hierarchy • Oracles • Connections to Complexity Theory • More Resources • Summary

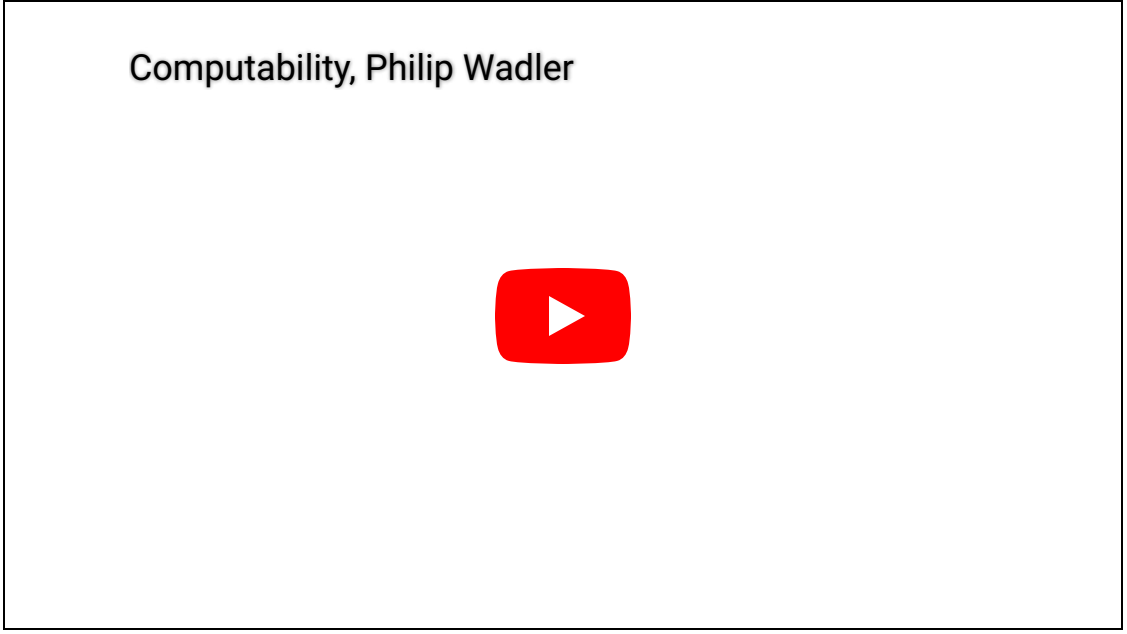
History

David Hilbert was good at formal logic. He wondered about many things.

He challenged mathematicians of his day with many problems.

Thinking there might be a way to algorithmically determine the truth or falsity of any statement in an axiomatic system, he posed the Entscheidungsproblem.

Philip Wadler tells the story:



Computation has limits. Mathematicians and programmers will never go out of business. Computer Science will always be an interesting field. We can dig deeper into this field of computation, and in particular, study what it means to be computable.

Models of Computation

We learned form Wadler that all three of these:

- Church's λ-calculus
- Gödel's μ-recursive functions
- Turing's Turing Machines

were equivalent. It turns out that a bunch of other efforts to formalize computation were also equivalent:

- Chomsky's Phrase-Structure Grammars
- String Rewriting Systems (also known as Semi-Thue Systems)
- Post's Post Production Systems
- All sorts of crazy extensions to Turing Machines (except magic oracles)

Because all of these formalisms are equivalent, and no one has found something better (other than magic oracles), most folks hypothesize that **we really have formally captured our intuitive notion of computability** with these things. The **Church-Turing Thesis** states that what we intuitively understand by "computation" is captured by the Turing Machine.

Limits of Computation?

In retrospect, one wonders *why* Hilbert suspected that every mathematical statement (i.e., a statement of a "formal axiomatic systems capable of representing arithmetic and beyond") was decidable! And thus, whether all true statements had proofs, or why there should exist a finite algorithm that could compute all functions in $\mathbb{N} \to \mathbb{N}$ (or even in $\mathbb{N} \to \mathbb{B}$).

For example, a simple diagonalization proof tells you there are non-computable functions in $\mathbb{N} \to \mathbb{N}$:

> **Proof**: Programs are finite strings of symbols so there are a countably infinite number of them. If all functions were computable, then there would need to be a program for each one, so there would need to be a countably infinite (enumerable) number of functions. If we *could* enumerate every function from $\mathbb{N} \to \mathbb{N}$ as $f_0, f_1, f_2, \ldots$, then the function f such that $f(n) = f_n(n) + 1$. would not be on the list. So the list is incomplete.

CLASSWORK

We're going to write out this **diagonalization** on the board.

***Exercise**: Prove that the number of functions from $\mathbb{N} \to \mathbb{B}$ is also uncountable.*

Okay nice, so there are more functions than programs. But are there any realistic functions we can specify that we cannot decide?

A First Non-Decidable Problem

Do there exist problems for which all instances have a YES-NO answer but no computer, no matter how powerful, nor any algorithm, in any formalism whatsoever, can compute the answer?

Of course, as we've shown above, a simple counting algorithm tells us that yes, they must exist! But are there any that we can describe and actually understand? Yes, and here's the classic first example. It is known as **The Halting Problem**:

> Write a function called `halts`, which accepts a function `f` and an input `x`, and returns `True` if `f(x)` halts, and `False` if `f(x)` enters an infinite loop when run.

Sounds simple enough. Let's **assume** we can write it:

```
def halts(f, x):
    # deciding the issue and return True or False
```

But here's the problem. Let's consider this, perfectly normal function:

```
def fox(f):
    if halts(f, f):
        while True: pass
```

The expression `fox(fox)` leads to an inconsistency: Running `fox(fox)` does this: "if `halts(fox, fox)` then go into an infinite loop, otherwise halt. Ugh. Wait. OMG. This cannot be. So our **assumption that `halts` could be written must be incorrect.** We cannot write `halts` in any programming language. The halting problem is undecidable.

> ***Python?***
>
> *It's way more fun to use Python in these proofs, but when we do, keep in mind these are isomorphic to language decision problems. In the Halting Problem case, we can re-cast this whole "problem" into one of deciding a particular language, namely:*
>
> $$\mathcal{H} = \{\langle M \rangle w \mid M \text{ halts on } w\}$$
>
> *where $\langle M \rangle$ is the string encoding of machine M.*

Recognizability

But guess what: Even though you can't **decide** whether a program will halt, you *can* recognize whether a program will halt. Just simulate it. So the language of halting programs is **recognizable but not decidable**.

If you can recognize a language, you *can* enumerate all its utterances. For concreteness, let's see how this is done with the Halting Problem. Since all programs are strings we can enumerate a list of all programs $p_0, p_1, p_2, p_3, \ldots$. We can do this:

- At time t_1, run one step of p_0
- At time t_2, run two steps of p_0 and one step of p_1
- At time t_3, run three steps of p_0, two steps of p_1, and one step of p_2
- ...

...and so on. Whenever a p_i halts, add it to your "output list" of halting programs.

This process enumerates all and only the halting programs.

We have the following fact:

> The set of all languages that are Turing-Recognizable is known as RE, namely, the set of **Recursively Enumerable Languages**.

and also:

> The set of all languages that are Turing-Decidable is known as R, namely, the set of **Recursive Languages**.

Beyond Recognizable

But now, what about the language of all strings that encode Turing Machines that never halt (sometimes called $\bar{H}$? That language isn't even recognizable. This will need a proof, but we'll get there soon.

So, how do we know a language isn't recognizable?

We can find a *first non-recognizable language* using diagonalization. The language is called $\mathcal{X}$ and is defined by:

$$\mathcal{X} = \{\langle M \rangle \mid M \text{ does not recognize } \langle M \rangle\}$$

(Informally, it is the set of all machines that don't recognize themselves.) To show this language is not recognizable by any Turing Machine let's make a list of all the Turing Machines there can possibly exist. There are a countably infinite number of these. We will call them $M_0, M_1, M_2, \ldots$. For each machine we will write out whether it recognizes the encodings of all of the machines:

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...

M_0	T	F	F	T	...
M_1	T	T	T	T	...
M_2	F	F	F	T	...
M_3	T	T	F	T	...
⋮	⋮	⋮	⋮	⋮	⋱

The T's in each row comprise the language recognized by M_i. The language $\mathcal{X}$ is the flipped diagonal of this table, which cannot appear in this table. No matter how we try to enumerate all the Turing Machines, there will never, EVER, be a row which makes $\mathcal{X}$. So $\mathcal{X}$ cannot be recognized by a Turing Machine.

Reductions

So the Halting Problem is undecidable. But how do we prove other problems undecidable? One way is to prove: "If problem P were decidable, then the Halting Problem would also be decidable."

This is done by **reducing** the Halting Problem to problem P.

In general, a **reduction** of problem A to problem B means we use an algorithm for B to solve A. You've probably done that a lot in the past, right? You've written "utility functions" and then called them to do a larger task.

Let's to this for the problem of determining whether $L(M) = \varnothing$. In formal notation, we want to know if we can decide:

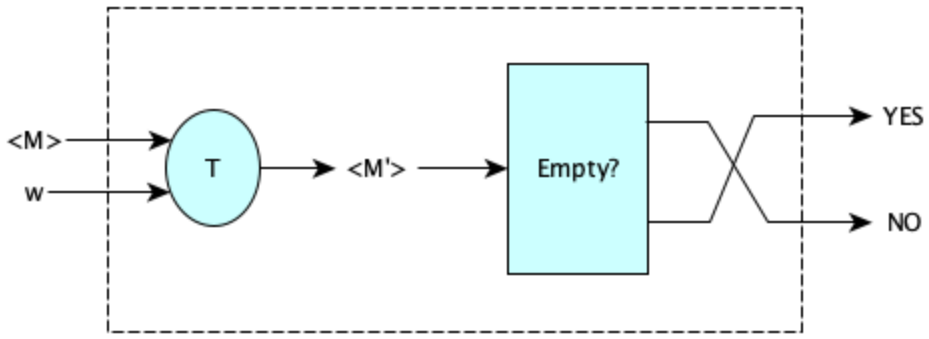
$$\mathcal{E} = \{\langle M \rangle \mid L(M) = \varnothing\}$$

This is the empty language problem. We can reduce the halting problem to it. Let's do it. Suppose $\mathcal{E}$ were decidable. This would mean we would have a machine E that when given the description of a machine M as input, would always halt with YES when $L(M) = \varnothing$ and halt with NO otherwise.

So let's make a machine called H that (1) takes two inputs, $\langle M \rangle$, the description of a Turing Machine, and w a string, and produces the description of a machine that erases its input, writes w on it, and simulates M, then (2) passes that new machine to E, then (3) swaps the outputs of E. Machine H

decides the halting problem. But this cannot be. The only place we could have gone wrong was our assumption that a decider for E exists. So $\mathcal{E}$ is undecidable.

A diagram might help:



Here $L(M') = \varnothing$ iff M does not halt on w, so the dashed box is a halting problem decider, which should not exist. So the Empty? machine cannot be a decider.

See Wikipedia's article on reducing problems and Wikipedia's list of undecidable problems.

Rice's Theorem

Here is something amazing.

Let's say you were given a general YES-NO problem about the run-time behavior of programs (e.g. does the program halt? or does this program ever output more than three symbols?). **Rice's Theorem** says that the problem is undecidable unless the property holds for all programs or holds for none.

More technically: All nontrivial semantic properties of programs are undecidable.

Here semantic means run-time and trivial means holds for all or holds for none.

More at Wikipedia.

> ***Exercise**: Research whether Rice's Theorem applies to languages as well. For example, the question "Is this language context-free?" is a YES-NO question. Clearly some languages are context-free and others are not. So the question is nontrivial. Does Rice's Theorem apply to this question?*

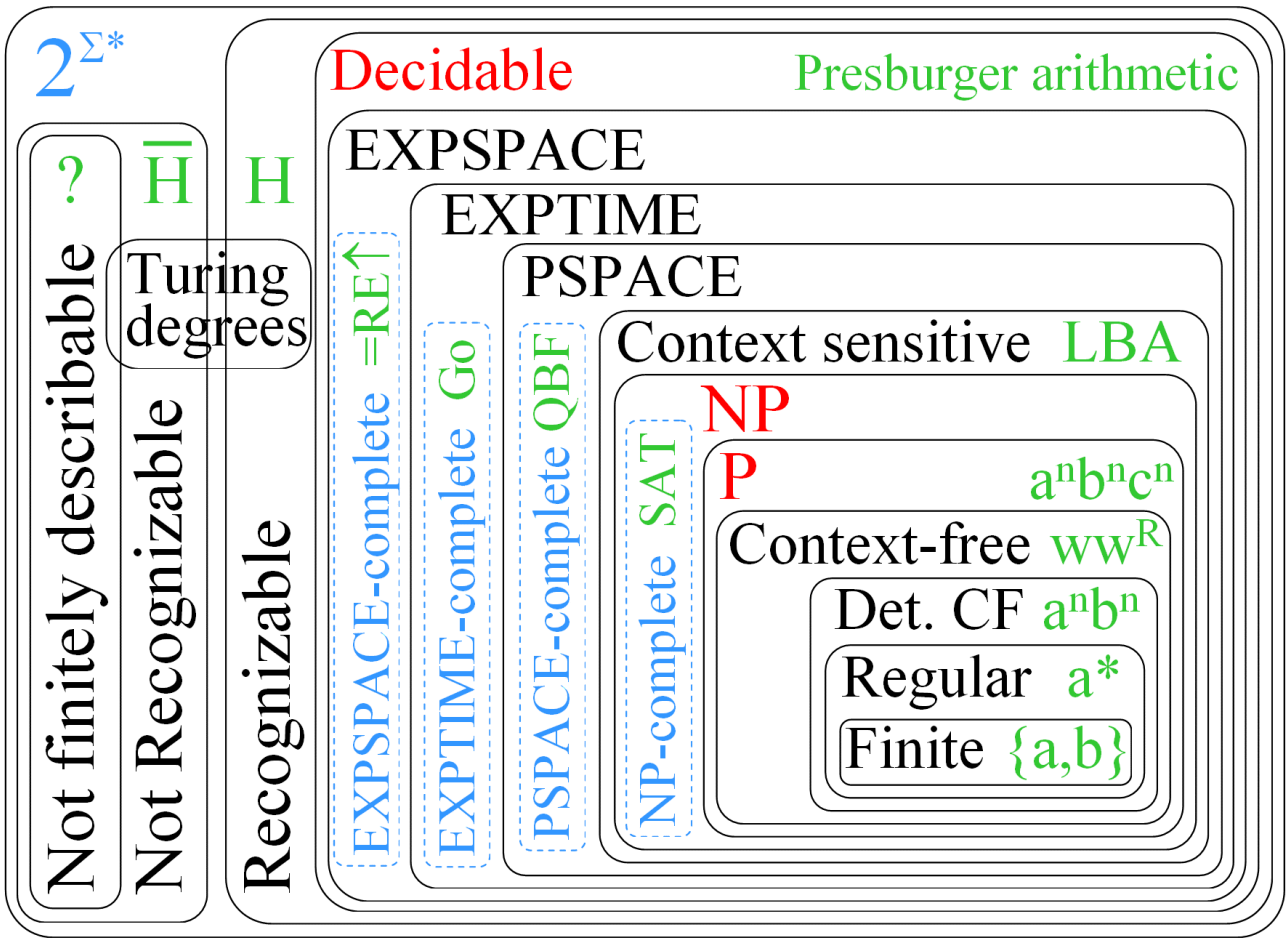
The Chomsky Hierarchy

The “limits” of computing models can define families of languages. For example:

The languages that can be recognized by	**are the ___ languages**	**which Chomsky called**
Turing Machines	Recognizable, a.k.a. Recursively Enumerable (RE)	Type 0
Turing Machines that always halt	Decidable, a.k.a. Recursive (R)	
LBA	Type-1 (CS)	Type 1
NPDA	Context-Free (CF)	Type 2
DPDA	Deterministic Context-Free (LR)	
FA	Regular (REG)	Type 3
FA with no cycles	Finite (FIN)	

ALL OF THESE LANGUAGE FAMILIES ARE PROPER SUBSETS OF THE FAMILY IN THE ROW ABOVE IT.

Gabe Robins has a beautiful visualization (which mixes in a few classes from complexity theory):



Oracles

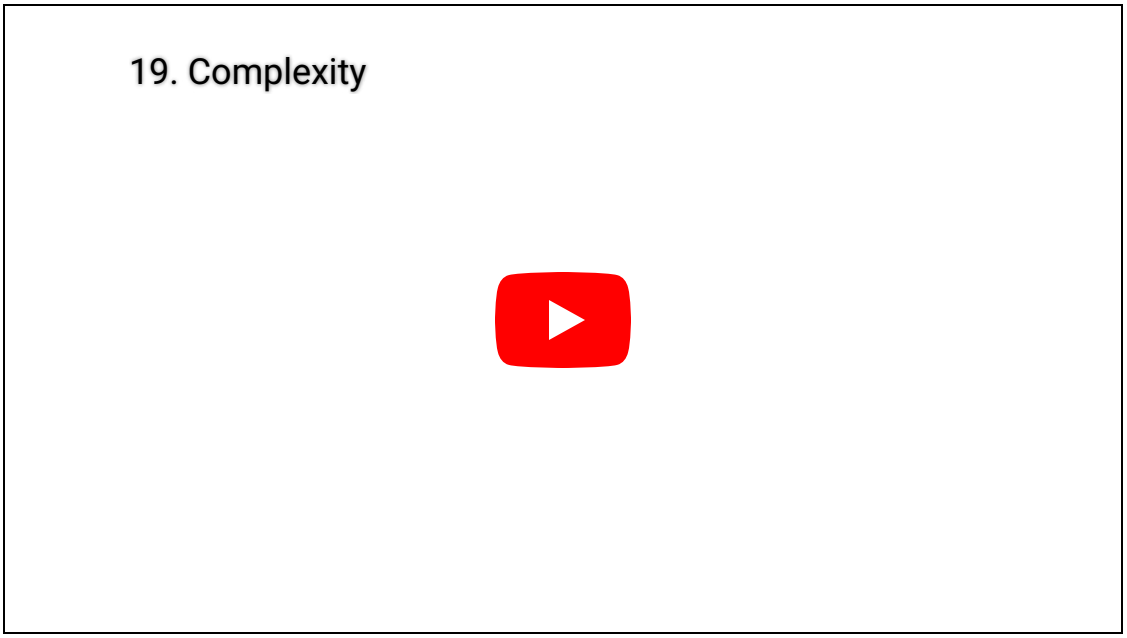
Loosely speaking, an **oracle** is something you can add to a machine, that, at any time during a computation, you can simple "ask the oracle" a question whose answer might not even be computable. It just magically answers the question and you go about your business.

Read about oracles at Wikipedia.

Connections to Complexity Theory

Computability Theory deals with what can and cannot be computed on a particular computing model. It does not make any claims on the number of steps required, or the amount of space required, to do the computation. That is what complexity theory is for.

But you know, maybe we *shouldn't* try to separate computability from complexity. These are convenient labels, but they are very inter-related. In fact, Robins's diagram intentionally places complexity classes in with the standard computability classes. Many schools offer a course called *Computability and Complexity*. Erik Demaine at MIT gives a brief overview of the limits of computation within his complexity theory lecture in his 6.006 class:



***Exercise**: Demaine expresses surprise that, and says it is depressing that, most problems are undecidable. Do you share that view? My personal view is one of wonder and amazement that we can actually know what the decidable problems are, given that virtually all problems are undecidable (in other words the "probability" that an arbitrary function is computable is zero). What is your take? Depression, or Awe? (Demaine does say we're "lucky" at least to know how to solve some problems, at least.)*

More Resources

You might also like:

- Gabe Robins's Theory Course Notes from University of Virginia
- Some nice notes from a Columbia course

When studying Robins's notes, focus on the set entitled "Formal languages, automata models, grammars, decidability, and Rice's theorem" (PDF here). There are several examples of problem reductions and of applications of Rice's Theorem.

Summary

We've covered:

- ✅ A history of computability theory
- ✅ Models of computation
- ✅ Limits of computation
- ✅ The halting problem
- ✅ The difference between recognizability and decidability
- ✅ The idea of problem reduction
- ✅ Rice's Theorem
- ✅ The Chomsky Hierarchy
- ✅ Oracles
- ✅ Connection to Complexity Theory

Regular Expressions



Regular Expressions are so cool. Knowledge of regexes will allow you to save the day.

CONTENTS

Definitions • Basic Examples • Notation • Using Regular Expressions • Components of Regexes • Performance Pitfalls • Performance Tips • Miscellaneous Language-Specific Notes • Study and Practice

Definitions

In formal language theory, a **regular expression** (a.k.a. regex, regexp, or r.e.), is a string that represents a regular (type-3) language.

Huh??

Okay, in many programming languages, a **regular expression** is a **pattern** that **matches** strings or pieces of strings. *The set of strings they are capable of matching goes way beyond what regular expressions from language theory can describe.*

Basic Examples

Rather than start with technical details, we'll start with a bunch of examples.

Regex	Matches any string that
`hello`	contains {hello}
`gray|grey`	contains {gray, grey}
`gr(a|e)y`	contains {gray, grey}
`gr[ae]y`	contains {gray, grey}
`b[aeiou]bble`	contains {babble, bebble, bibble, bobble, bubble}
`[b-chm-pP]at|ot`	contains {bat, cat, hat, mat, nat, oat, pat, Pat, ot}
`colou?r`	contains {color, colour}
`rege(x(es)?|xps?)`	contains {regex, regexes, regexp, regexps}
`go*gle`	contains {ggle, gogle, google, gooogle, goooogle, ...}
`go+gle`	contains {gogle, google, gooogle, goooogle, ...}
`g(oog)+le`	contains {google, googoogle, googoogoogle, googoogoogoogle, ...}
`z{3}`	contains {zzz}
`z{3,6}`	contains {zzz, zzzz, zzzzz, zzzzzz}
`z{3,}`	contains {zzz, zzzz, zzzzz, ...}
`[Bb]rainf\*\*k`	contains {Brainf**k, brainf**k}
`\d`	contains {0,1,2,3,4,5,6,7,8,9}
`\d{5}(-\d{4})?`	contains a United States zip code
`1\d{10}`	contains an 11-digit string starting with a 1
`[2-9]|[12]\d|3[0-6]`	contains an integer in the range 2..36 inclusive
`Hello\nworld`	contains Hello followed by a newline followed by world
`mi.....ft`	contains a nine-character (sub)string beginning with mi and ending with ft (Note: depending on context, the dot stands either for “any character at all” or “any character except a newline”.) Each dot is allowed to match a different character, so both microsoft and minecraft will match.
`\d+(\.\d\d)?`	contains a positive integer or a floating point number with exactly two characters after the decimal point.

`[^i*&2@]`	contains any character other than an i, asterisk, ampersand, 2, or at-sign.
`//[^\r\n]*[\r\n]`	contains a Java or C# slash-slash comment
`^dog`	begins with "dog"
`dog$`	ends with "dog"
`^dog$`	is exactly "dog"

Notation

There are many different syntaxes for regular expressions, but in general you will see that:

- Most characters stand for themselves
- Certain characters, called metacharacters, have special meaning and **must be escaped** (usually with `\`) if you want to use them as characters. In most syntaxes the metacharacters are:

  `(` `)` `[` `]` `{` `}` `^` `$` `.` `\` `?` `*` `+` `|`

- Within square brackets, you only have to escape (1) an initial `^`, (2) a non-initial or non-final `-`, (3) a non-initial `]`, and (4) a `\`.

Using Regular Expressions

Many languages allow programmers to define regexes and then use them to:

- **Validate** that a piece of text (or a portion of that text) matches some pattern
- **Find** fragments of some text that match some pattern
- **Extract** fragments of some text
- **Replace** fragments of text with other text

Generally a regex is first **compiled** into some internal form that can be used for super fast validation, extraction, and replacing. Sometimes there is an explicit `compile` function or method, and sometimes special syntax is used to compile, such as the very common form `/.../`.

Validation

Example: find `"color"` or `"colour"` in a given string.

```
// Java
Pattern p = Pattern.compile("colou?r");
Matcher m = p.matcher("The color green");
m.find();                           // returns true
m.start();                          // returns 4
m.end();                            // returns 9
m = p.matcher("abc");
m.find();                           // returns false


# Perl
$p = /colou?r/;
"The color green" =~ $p;            # returns 1 (cuz no Perl true)
"abc" =~ $p;                        # returns 0 (cuz no Perl false)


# Ruby
p = /colou?r/
"The color green" =~ p              # returns 4
"abc" =~ p                          # returns nil


# Python
p = re.compile("colou?r")
m = p.search("The color green")
m.start()                           # returns 4
m = p.search("abc")                 # returns None


// JavaScript
const p = /colou?r/;
"The color green".search(p);        // returns 4
"abc".search(p);                    // returns -1
```

If you want to know if an entire string matches a pattern, define the pattern with `^` and `$`, or with `\A` and `\Z`. In Java, you can call `matches()` instead of `find()`.

Extraction

After doing a match against a pattern, most regex engines will return you a bundle of information, including such things as:

- the part of the text that matched the pattern
- the index within the string where the match begins
- each part of the text matching the parenthesized portions within the pattern
- (sometimes) the text before the matched text
- (sometimes) the text after the matched text

Example in Ruby:

```
>> pattern = /weighs (\d+(\.\d+)?) (\w+)/
=> /weighs (\d+(\.\d+)?) (\w+)/
>> pattern =~ 'The thing weighs 2.5 kilograms or so.'
=> 10
>> $&
=> "weighs 2.5 kilograms"
>> $1
=> "2.5"
>> $2
=> ".5"
>> $3
=> "kilograms"
>> $`
=> "The thing "
>> $'
=> " or so."
```

The same thing in JavaScript:

```
> const pattern = /weighs (\d+(\.\d+)?) (\w+)/
> pattern.exec('The thing weighs 2.5 kilograms or so.')
[ 'weighs 2.5 kilograms',
  '2.5',
  '.5',
  'kilograms',
  index: 10,
  input: 'The thing weighs 2.5 kilograms or so.' ]
```

Note how in JavaScript, the match result object looks like an array and an object.

The so-called *group numbers* are found by counting the left-parentheses in the pattern:

TODO PICTURE GOES HERE

Sometimes you need parentheses only for precedence purposes and you don't want to incur the cost of extracting a group. We have **non-capturing groups** for this purpose.

Ruby:

```
>> phone = /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/
=> /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/
>> phone =~ 'Call 555-1212 for info'
=> 5
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["Call ", "555-1212", " for info", nil, nil, "555", "1212", nil]
>> phone =~ '800.221.9989'
=> 0
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["", "800.221.9989", "", "800.", "800", "221", "9989", nil]
>> phone =~ '1800.221.9989'
=> 1
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["1", "800.221.9989", "", "800.", "800", "221", "9989", nil]
```

JavaScript:

```
> const r = /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/g;
> const m = r.exec("Call 1.800.555-1212 for info");
> m.index
7
> JSON.stringify(m);
["800.555-1212","800.","800","555","1212"]
```

Java

```
Pattern phone = Pattern.compile("(((\\d{3})(?:\\.|-))?(\\d{3})(?:\\.|-)(\\d{4})");
String[] tests = {"Call 555-1212 for info", "800.221.9989", "1800.221.9989"};
for (String s : tests) {
    Matcher m = phone.matcher(s);
    m.find();
    System.out.println("groupCount = " + m.groupCount());
    System.out.println("group(0) = " + m.group(0));
    System.out.println("group(1) = " + m.group(1));
    System.out.println("group(2) = " + m.group(2));
    System.out.println("group(3) = " + m.group(3));
    System.out.println("group(4) = " + m.group(4));
}
```

```
groupCount = 4
group(0) = 555-1212
group(1) = null
group(2) = null
group(3) = 555
group(4) = 1212
groupCount = 4
group(0) = 800.221.9989
group(1) = 800.
```

```
group(2) = 800
group(3) = 221
group(4) = 9989
groupCount = 4
group(0) = 800.221.9989
group(1) = 800.
group(2) = 800
group(3) = 221
group(4) = 9989
```

Substitution

Many languages have `replace` or `replaceAll` methods that replace the parts of a string that match a regex. Sometimes you will see a `g` flag on a regex instead of a `replaceAll` function.

```
alert("Rascally Rabbit".replace(/[RrLl]/g, "w"));
```

Components of Regexes

Character Classes

- Square brackets `[ ]` — means exactly one character
- A leading `^` negates, a non-leading, non-terminal `-` defines a range:

```
[abc]          a or b or c
[^abc]         any character _except_ a, b, or c (negation)
[a-zA-Z]       a through z or A through Z, inclusive (range)
```

- If you have a `]` in your set, put it first. Use `\` to escape.
- Java allows crazy extensions:

```
[a-d[m-p]]     [a-dm-p] (union, Java only, I think)
[a-e&&[def]]   [de] (intersection, Java only, I think)
[a-r&&[^bq-z]] [ac-p] (subtraction, Java only, I think)
```

- Other ways to say exactly one character from a set are:

```
\d              [0-9]
\D              [^\d]
\s              [ \t\n\x0B\f\r]
\S              [^\s]
\w              [a-zA-Z0-9_]
\W              [^\w]
.               any character at all, except maybe not a line
```

Groups

Defined above, in the section on extraction.

Quantifiers

Generally, 18 types:

	Eager	**Reluctant**	**Possessive**
Zero or one	`?`	`??`	`?+`
Zero or more	`*`	`*?`	`*+`
One or more	`+`	`+?`	`++`
m times	`{m}`	`{m}?`	`{m}+`
At least m times	`{m,}`	`{m,}?`	`{m,}+`
At least m, at most n times	`{m,n}`	`{m,n}?`	`{m,n}+`

Eager (Greedy and Generous) — match as much as possible, but give back

```
\w+\d\d\w+  // matches abcdef42skjhfskjfhsjdfs
            // but inefficiently
```

Possessive — match as much as possible, but do NOT give back

```
\w++\d\d\w+  // does not match abcdef42skjhfskjfhsjdfs
             // but is efficient
```

Reluctant — match as little as possible

```
\w+?\d\d\w+  // matches abcdef42skjhfskjfhsjdfs
             // efficiently, yay!
```

Backreferences

Things captured can be used later:

```
<(\w+)>[^<]*</\1>
```

Anchors, Boundaries, Delimiters

Some regex tokens do not consume characters! They just assert the matching engine is at a particular place, so to speak.

- `^` : Beginning of string (or line, depending on the mode)
- `$` : End of string (or line, depending on the mode)
- `\A` : Beginning of string
- `\z` : End of string
- `\Z` : Varies a lot depending on the engine, so be careful with it
- `\b` : Word boundary
- `\B` : Not a word boundary

Read more about these at Rexegg.

Also, the lookarounds (up next!) don't consume any characters either!

Lookarounds

- Lookarounds do not consume anything

- Even though they have parens, they do not capture
- **Positive Lookahead**: Matches only if followed by something

```
Hillary(?=\s+Clinton)
```

matches the Hillary in Hillary Clinton but not the Hillary in Hillary Makasa.

- **Negative Lookahead**: Matches only if not followed by something

```
q(?!u)
\b(?:[a-eg-z]|f(?!oo))\w*\b      // Word not starting with foo
\b(?:[a-eg-z]|f(?!oo\b))\w*\b    // Any word except foo
((?!foo).*)
```

- **Positive Lookbehind**: Matches only if preceded by something

```
(?<=-)\p{L}+              // a word following a hyphen
(?<=http://)\S+           // URL not including the http:// part
```

- **Negative Lookbehind**: Matches only if not preceded by something

```
(?<![-+\d])(\d+)      // Digits not preceded by a digit, +, or -
```

- Lookarounds show up in search and replace applications

```
Pattern p = Pattern.compile("Hillary(?=\\s+Clinton)");
String text = "Once Hillary Clinton was talking about Sir\n" +
    "Edmund Hillary to Hillary Makasa and then Hillary\n" +
    "Clinton had to run off on important business.";
Matcher m = p.matcher(text);
System.out.println(m.replaceAll("Secretary"));
```

Note: Read this awesome article on lookarounds.

Modifiers

A modifier affects the way the rest of the regex is interpreted. Not every language supports all of the modifiers below. For example, JavaScript (officially) supports only i, g, and m.

Modifier	Meaning
`g`	global
`i`	ignore case
`m`	multiple line
`s`	single line (DOTALL): Means that the dot matches any character at all. Without this modifier, the dot matches any character except a newline.
`x`	ignore whitespace in the pattern
`d`	Unix line mode: Considers only U+000A as a line separator, rather than U+000D or the U+000D/U+000A combo or even U+2028.
`u`	Unicode case: in this mode the case-insensitive modifier respects Unicode cases; outside of this mode that modifier only consolidates cases of US-ASCII characters.

Performance Pitfalls

You should know some things about how your *regex engine* works since two "equivalent" regexes can have drastic differences in processing speed.

- It is possible to write regexes that take exponential time to match, but you pretty much have to TRY to make one (they're pathological)
- It is more common to accidentally create regexes that run in quadratic time
- Main types of problems
    - Recompilation (from forgetting to compile regexes used multiple times)

      ```
      // Java shortcut, should not be used in most cirumstances
      s.matches("colou?r");
      ```

    - Dot-star in the Middle (which causes backtracking)
        - Solution 1: Use negated character class
        - Solution 2: Use reluctant quantifiers
    - Nested Repetition

Performance Tips

Always do the following:

- Use non-capturing groups when you need parentheses but not capture.
- If the regex is very complex, do a quick spot-check before attempting a match, e.g.
  - Does an email address contain '`@`'?
- Present the most likely alternative(s) first, e.g.
  - `black|white|blue|red|green|metallic seaweed`
- Reduce the amount of looping the engine has to do
  - `\d\d\d\d\d` is faster than `\d{5}`
  - `aaaa+` is faster than `a{4,}`
- Avoid obvious backtracking, e.g.
  - `Mr|Ms|Mrs` should be `M(?:rs?|s)`
  - `Good morning|Good evening` should be `Good (?:morning|evening)`

Miscellaneous Language-Specific Notes

A few things that are good to know:

- Java's built-in support for regexes exceeds that of many languages
- Especially good for Unicode and for character classes (has more than Perl)
- Syntax is more cumbersome (string literal support weak, no operator for matching...) — live with it!
- Perl has nice regex, too, even allows you can even embed code inside them.
- Great Perl documentation at the perldoc pages perlrequick, perlretut, perlre. and perlreref.
- JavaScript seems to less extensive support than other languages, but I think this is changing.
- Python puts regex functions in a module.
- Python docs are here.

Study and Practice

Here are some good sources:

- The Premier Site for Regexes
- Maybe this is a better premier site
- Regex 101—Develop regexes online
- RegExr (Awesome online tool for Java regexes)
- Rubular (Ruby Online Regex Tester)
- Perl Regular Expressions Tutorial
- Perl Regular Expressions (Manual)
- Perl Regular Expressions Quick Reference
- Ruby Regexp class documentation
- Java Regex Tutorial
- Java Regex Optimization Article
- Java Pattern class API docs
- JavaScript Regular Expressions (at Mozilla Developer Center)
- Python Regular Expressions

Code Improvement

People like to write efficient and compact code. Let's try to get a compiler to write this kind of code, too.

CONTENTS

Motivation and Goals • Where Can Optimization Be Done? • Optimization Strategies • A Catalog of Optimization Techniques • Links • Examples • Further Reading • Summary

Motivation and Goals

We want a compiler to produce *good* code, at least we usually do. We may wish to "optimize" one or more of:

- Program size
- Program speed
- Resource usage (page swaps, caches, file handles, threads, etc.)

General **optimization** is undecidable, and many specific optimization tasks happen to take provably exponential time or are NP-Hard. So we normally speak of code **improvement**.

The general rule is to do those things that give the **most** improvement for the **least** amount of work, so for instance, you might see compilers that don't bother doing any improvement outside loops. Still, there are different degrees of optimization aggressiveness:

- No improvements at all
- Only few basic simplifications on AST nodes or tuples (peephole optimization)
- Attempt near-optimal code **within basic blocks** (by removing redundancies like common subexpressions, doing smart instruction scheduling and register allocation, etc.)
- **Intraprocedural optimization** (improving across basic blocks, but still within a function, e.g., loop optimization using data flow analysis)
- **Interprocedural optimization**

Ideally, optimizations should satisfy two criteria:

SAFETY

The optimized code does the “same thing” as before

PROFITABILITY

The optimized code is actually smaller or faster

but as we’ll see, unsafe and unprofitable optimizations are sometimes done in practice.

Where Can Optimization Be Done?

There is no single optimization phase in a compiler. Lots of things happen to improve code:

1. The programmer needs to find a better algorithm. Not many (if any) compilers can turn bubblesort into quicksort.
2. The intermediate code generator can be smart about how it generates intermediate code.
3. The compiler can make a pass or two or three over the intermediate code to improve it.
4. The code generator can be smart, with clever instruction selection, by employing register tracking, etc.
5. The compiler can make a pass or two or three over the target code to improve it.
6. The run time system can adapt to the way a program is running, possibly recompiling and relinking on the fly.
7. The programmer profiles the running code and finds out where the memory leaks and bottlenecks and excessive resource consumption are occurring, and rewrites the code accordingly.

Optimization Strategies

In general, you’ll try to:

- Use cheaper, more efficient operations
- Reuse results rather than recompute them

- Maximize the use of registers, caches, and memory
- Reduce the amount of code (e.g., remove dead and useless code)
- Reduce conditional jumps (they make branch prediction difficult)
- Optimize the use of pipelines and cores

But note the need for tradeoffs! You can get rid of some jumps by unrolling loops, but this makes more code.

> ***Exercise**: What are other conflicts?*

A Catalog of Optimization Techniques

Machine Independent Optimizations

- Constant Folding
- Algebraic Simplification (e.g. operand reordering)
- Strength Reduction
- Unreachable Code Elimination
- Dead Code Elimination
- Copy Propagation
- Common Subexpression Elimination
- Loop Unrolling
- Loop Fission
- Loop Fusion
- Loop Inversion
- Loop Interchange
- Loop Invariant Factoring
- Jump Threading
- Induction Variable Simplification
- Tail Recursion Elimination
- Inline Expansion

Mostly Machine Independent Optimizations

- Efficient address computation
- Simplified stack frames
- Static allocation of frames
- Inlining

Machine Dependent Optimizations

- Careful instruction selection, using an understanding of caches, pipelines, branch prediction algorithms, scheduling rules, alignment requirements
- Smart register allocation
- Smart use of address modes
- Balancing across multiple CPUs

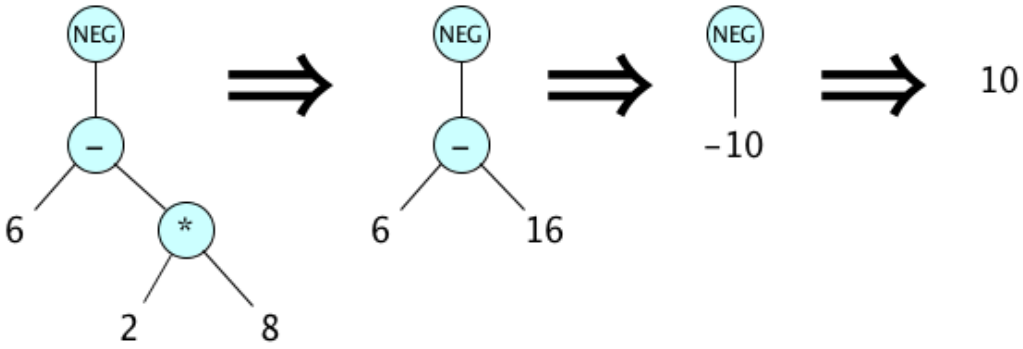
Links

- Compiler Optimization at Wikipedia
- Wikipedia's Compiler Optimizations Category Page
- Agner Fog's Software Optimization Resources
- A pretty good reference

Examples

Constant Folding

Why wait until the target program is run to do simple computations? Let the compiler do it. Example:



Remember we can always write trees as strings, so we'll do that from now on:

`(NEG (– 6 (* 2 8)))`	`(NEG (– 6 16))`	`(NEG –10)`	`10`

Strength Reduction

You can replace expensive operations with cheaper ones:

`(+ x 0)`	`x`
`(- x 0)`	`x`
`(- x x)`	`0`
`(- 0 x)`	`(NEG x)`
`(+ x (NEG y))`	`(- x y)`
`(- x (NEG y))`	`(+ x y)`
`(* x 0)`	`0`
`(* x 1)`	`x`
`(/ x 1)`	`x`
`(/ x x)`	`1`
`(% x 1)`	`0`
`(% x x)`	`0`
`(/ 0 x)`	`0`
`(% 0 x)`	`0`
`(< x x)`	`false  -- also for > and NOTEQ`
`(EQ x x)`	`true  -- also for <= and >=`
`(AND x false)`	`false`
`(OR x true)`	`true`
`(NOT <= x y)`	`(> x y)`
`(>= (ABS x) 0)`	`true`
`(* x 4)`	`(SHL x 2)`
`(/ x 4)`	`(SHR x 2)  -- arithmetic right shift`

***Exercise**: Think up at least a dozen more of these.*

You can even "invent" some powerful operations:

`(+ (* x 90) y)`	`(MULADD x 90 y)`
`(AND (<= x 3) (>= x -10))`	`(INRANGE x -10 3)`

> *By "inventing" powerful operations, we mean creating new AST nodes representing operations that may not appear in your source language. That is, a language may not have INC, MULADD, INRANGE or a host of other operators, but during optimization there is nothing wrong with adding these new nodes! In fact, some can be created for the sole purpose of allowing better code to be generated.*

Operand Reordering

Use commutativity, associativity, and distributivity to either reduce the number of operations, or do things like "put the more complex code on the left:"

`(+ x (- y z))`	`(+ (- y z) x)`	`-- commutativity`
`(* x (* y z))`	`(* (* y z) x)`	`-- associativity`
`(- (* x 5) (* y 5))`	`(* (- x y) 5)`	`-- distributivity`

Complex on the left simplifies register allocation on some machines.

But watch out: associativity and distributivity changes may not preserve overflow semantics.

```
$ swift
  1> let x: Int8 = 70
x: Int8 = 70
  2> let y: Int8 = 80
y: Int8 = 80
  3> let z: Int8 = 30
z: Int8 = 30
  5> (x + (y - z))
$R0: Int8 = 120
  4> (x + y) - z
Execution interrupted. Enter code to recover and continue.
Enter LLDB commands to investigate (type :help for assistance.)
```

Assignment Simplification

As long as assignment doesn't have hidden side-effects, you can do things like this:

`(ASSIGN x x)`	`(NOP)`
`(ASSIGN x (+ x 1))`	`(INC x)`
`(ASSIGN x (- x 1))`	`(DEC x)`
`(ASSIGN x (+ x y))`	`(+= x y)`

> *Beware of languages that allow for side effects or triggers to take place on assignment. For example Ada allows assignment triggers, and C++ allows you to overload the assignment operator. The optimzations above prevent those effects and triggers, hence those optimizations are strictly speaking unsafe.*

Unreachable Code Elimination

If you can't ever get to some code, don't generate it. With trees:

`(IF true T1 T2)`	`T1`

`(IF false T1 T2)`	`T2`
`(BLOCK T1 T2 RETURN T3 T4)`	`(BLOCK T1 T2 RETURN)`
`(WHILE false T)`	`(NOP)`
`(FOR i emptyrange T)`	`(NOP)`

Other general strategies:

- Remove all tuples after an unconditional control transfer (JUMP, RETURN, THROW), up to, but not including, the next LABEL.
- Remove all basic blocks with no predecessors
- Remove all private functions or methods that are never called

Dead Code Elimination

If a local variable or by-value parameter is assigned to but never subsequently used, you can delete the assignment. As usual, do this only if no implicit operations have side effects, and watch out for aliasing.

```
void f() {
    int p = 5;
    int* q = &p;
    p = 3;          // LOOKS LIKE DEAD CODE BUT IT IS NOT!!
    cout << *q;
}
```

Copy Propagation

After an assignment `x := y` (i.e., `(COPY y x)`), replace all *uses* of `x` with `y` until such point that the values of `x` or `y` could be different. This might turn the copy into dead code which you can eliminate. Given the source code:

```
w := 4;
a = w - 7 + (z * w)
```

then optimizing the AST proceeds as follows:

`(BLOCK`
`  (ASSIGN w 4)`
`  (ASSIGN a`
`  (+ (- w 7) (* z w))`
`)`
	`(ASSIGN a`
	`  (+ (- 4 7) (* z 4)))`
		`(ASSIGN a`
		`  (+ (- 3) (* z 4))`
			`(ASSIGN a`
			`  (- (* z 4) 3))`

And with Tuples:

(COPY,4,w) (MUL,z,w,t1) (SUB,w,7,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (SUB,4,7,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (COPY,-3,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (ADD,-3,t1,a)	(MUL,z,4,t1) (SUB,t,3,a)

Common Subexpression Elimination

If an expression appears multiple times, **AND ITS VALUE CANNOT CHANGE IN BETWEEN THESE EVALUATIONS**, then you can just evaluate it once:

`x * 3 + x * 3 + 10`	`_t = x * 3;` `_t + _t + 10;`	
`a[i][j] = a[i][j] + 1`	`_t = &a[i][j];` `(*_t)++;`	

`x = a[i];`
`a[i] = a[j];`
`a[j] = x;`
	`_t0 = &a[i];`
	`x = *_t0;`
	`_t1 = &a[j];`
	`*_t0 = *_t1;`
	`*_t1 = x;`
		`--After CP and DCE`
		`_t0 = &a[i];`
		`_t1 = &a[j];`
		`*_t0 = *_t1;`
		`*_t1 = *_t0;`

Expressions that look common may not be, due to pointers, aliasing, references, arrays, etc.

```
x = a[i] - 85;
a[j] = 21;          // a[i] is NOT common
print(a[i]);        // ... because maybe i == j
```

If your language has references, be very careful

```
x = j * 8 - y;
q = 10;             // Make sure you never had done
c = j * 8 - y       // ... something like int& q = j
```

Conditional Jump Tuple Compression

A naive translation into tuples for:

```
if (x < 3) { ... }
```

can be optimized like so:

(LT, x, 3, t0) (JZ, t0, L3)	(JGE, x, 3, L3)

This can be very useful since the naive translation of the first tuple sequence to x86 assembly would be:

```
cmp x, 3
setl dl
movzx eax, dl
test eax, eax
jz L3
```

and for the second:

```
cmp x, 3
jg3 L3
```

Loop Unrolling

Loops can have a performance issue, since time is required to execute those jumps, and conditional jumps can cause slowdowns because of the potential for branch misprediction.

`for i in 1..4 {`
`  T`
`}`
	`i = 1; T;`
	`i = 2; T;`
	`i = 3; T;`
	`i = 4; T;`

`(FOR i (RANGE 1 4) T)`	`(BLOCK`
	`  (ASSIGN I 1) T`
	`  (ASSIGN I 2) T`
	`  (ASSIGN I 3) T`
	`  (ASSIGN I 4) T`
	`)`

Loop unrolling is commonly followed by copy propagation!

`for i in 1...4 {`
`  print(i * 2)`
`}`
	`i = 1; print(i * 2);`
	`i = 2; print(i * 2);`
	`i = 3; print(i * 2);`
	`i = 4; print(i * 2);`
		`print(1 * 2);`
		`print(2 * 2);`
		`print(3 * 2);`
		`print(4 * 2);`
			`print(2);`
			`print(4);`
			`print(6);`
			`print(8);`

Look unrolling might conflict with cache issues, since the code is getting bigger.

Note: you can *partially unroll*, that works too.

Loop Fusion

Suppose you know that you had an array of structs:

```
struct {
  int a;
  int b;
} s[5000];
```

then two loops accessing the a's and b's independently could be **fused** into one to improve locality (and avoid destroying performance due to cache unloading and refilling:

`for i in 0..<n {`
`  ... s.a[i] ...`
`}`
`for i in 0..<n {`
`  ... s.b[i] ...`
`}`
	`for i in 0..<n {`
	`  ... s.a[i] ...`
	`  ... s.b[i] ...`
	`}`

Loop Fission

Suppose you know that you had two parallel arrays in a struct:

```
struct {
  int a[5000];
  int b[5000];
} s;
```

then one loop that iterated both array could be **split** into two loops accessing the a's and b's independently to improve locality (and avoid destroying performance due to cache unloading and refilling:

`for i in 0..<n {`
`  ... s.a[i] ...`
`  ... s.b[i] ...`
`}`
	`for i in 0..<n {`
	`  ... s.a[i] ...`
	`}`
	`for i in 0..<n {`
	`  ... s.b[i] ...`
	`}`

Loop Inversion

This interesting technique replaces a while loop with a do-while loop in an if statement, looking like this at the source code view (BUT DON'T REALLY DO THIS IN YOUR SOURCE CODE, LET THE COMPILER DO IT):

`while (condition) {`
`  body`
`}`
	`if (condition) {`
	`  do {`
	`  body`

```
  } while (condition);
}
```

This tends to generate better machine code, because there are fewer jumps. Here it is in tuples:

```
top:
  condition
  (JZ, bottom)
  body
  (JUMP top)
bottom:
  ...
```

```
  condition
  (JZ, bottom)
top:
  body
  condition
  (JNZ, top)
bottom:
  ...
```

The last iteration of the loop has two jumps not taken in the optimized version compared to the unoptimized version. Pretty useful if this is a nested loop!

For a loop executed n times	Unoptimized Version			Optimized Version	
	Conditional Jumps		Unconditional Jumps Taken	Conditional Jumps	
	Taken	Fallen Through		Taken	Fallen Through
0	1	0	0	1	0
1	1	1	1	0	2
2	1	2	2	1	2
3	1	3	3	2	2
4	1	4	4	3	2
5	1	5	5	4	2
100	1	100	100	99	2

> ***Remember these are *compiler* optimizations***
>
> *Don't use the examples on this page as an example of how you, as a programmer, should be writing source code. The page discusses optimizations that a compiler can do for you. You (the programmer) are supposed to write clear, readable code. Let the compiler optimize. It's generally much better at that you are.*

Loop Interchange

If you know that your matrices are laid out in row-major (as in C) or column-major (as in Fortran) order, and you detect that a matrix is being processed with nested loops in a manner that is opposite from the layout, and you can tell that the body of the loop is not modifying the matrix in any way dependent on the traversal order, you can switch the traversal order. Here is an example of switching from row order to column order:

```
for i in 0..<m {
    for j in 0..<n {
        ... a[i, j] ...
    }
}
```

```
for j in 0..<n {
  for i in 0..<m {
      ... a[i, j] ...
  }
}
```

Loop Invariant Factoring

A **loop invariant** is an expression that is always the same for every execution of the loop. You might consider moving the evaluation of this expression just outside (just before) the loop, so that it evaluated only once. Example:

```
for i in 0..<n {
    for j in 0..<n {
        for k in 0..<n {
            a[i][j][k] = i * j * k
        }
    }
}
```

```
for i in 0..<n {
    _t0 = a[i]
    for j in 0..<n {
        _t1 = _t0[j]
        _t2 = i * j
        for k in 0..<n {
            _t1[k] = _t2 * k
        }
    }
}
```

If t = 100, for example then:

- Without factoring: 3,000,000 array accesses and 2,000,000 multiplications
- With factoring: 1,010,100 array accesses and 1,010,000 multiplications

> ***Factoring may be unsafe***
>
> *You may factor an expression that throws an exception (or crashes), and if it does the exception would be thrown at the wrong place, perhaps before a side-effect which could completely change the program meaning.*
>
> *A good compiler will only factor expressions it knows cannot throw.*

> ***Factoring may be unprofitable as well as unsafe***

Factoring an invariant from a loop that is executed zero times causes unnecessary and potentially harmful execution of code that should not ever have been run.

Jump Threading

Sometimes a jump will land on a jump (or a throw or a return). Examples (for tuples):

`(JGE, x, 3, L5)`
`some code`
`L5:`
`(JUMP, L7)`
	`(JGE, x, 3, L7)`
	`some code`
	`L5:`
	`(JUMP, L7)`

`(JGE, x, 3, L5)`
`some code`
`L5:`
`(JGE, x, 2, L7)`
	`(JGE, x, 3, L7)`
	`some code`
	`L5:`
	`(JGE, x, 2, L7)`

Here's a nice Jump Threading article.

The basic idea of jump threading is to avoid conditional jumps that you know will always be taken or never be taken, and make sure they are never executed. The same theme appears in transformations such as these (at the AST level):

`(BLOCK` `(IF c T1) (IF c T2))`	`(IF c (BLOCK T1 T2))`
`(BLOCK` `(IF c T1) (IF (NOT c) T2))`	`(IF c T1 T2)`

Again, only perform these if you can prove execution of T1 will not affect c.

Induction Variable Simplification

An **induction variable** is any variable whose value is incremented by a constant each pass through the loop.

An **induction expression** has the form `i*c1+c2` for induction variable `i` and invariants `c1` and `c2`. You can get rid of the multiplication that it is the loop, replacing it with a much cheaper addition! Here's an example:

`for i in 1...n {`
`a[i] = 80 * i + q`
`}`
	`_t = 80 + q`
	`for i in 1...n {`
	`a[i] = _t`
	`_t = _t + 80`
	`}`

In general, replace induction expression `i*c1+c2` by initializing a new variable to `i0*c1+c2` and incrementing by `s*c1` where `s` is the step size

> ***Exercise:*** *But is this okay with floating point numbers that require high precision?*

Tail Recursion Elimination

A function that returns a call to itself can be transformed into a loop. The classic gcd example is:

```
function gcd(x, y) {
  if (y === 0) {
    return x;
  }
  return gcd(y, x % y);
}
```

```
function gcd(x, y) {
  while (true) {
    if (y === 0) {
      return x;
    }
    [x, y] = [y, x % y];
    continue;
  }
}
```

The `continue` is emitted just in case there is more code after the return, but of course if the optimizer can tell there is no code there, then it is unnecessary. Be careful that the `return` is not inside a nested loop!

With tuples, the transformation is like:

```
(JNZ, y, L2)
(RET, x)
(LABEL, L2)
(STARTCALL, gcd)
(PARAM, y)
(MOD, x, y, t0)
(PARAM, t0)
(CALL, gcd, t1)
(RET, t1)
```

```
(LABEL, L1)          -- add this new label at the top
(JNZ, y, L2)
(RET, x)
(LABEL, L2)
(COPY, y, t0)        -- compute new first param
(MOD, x, y, t1)      -- compute new second param
(COPY, t0, x)        -- load new first param
(COPY, t1, y)        -- load new second param
(JUMP, L1)           -- "call" by jumping to top
```

For fun, here are the tail recursion elimination applications of Factorial (remember a should be primed with 1 here):

```
function fact(n, a) {
  if (n === 0) {
    return a;
  }
  return fact(n - 1, a * n);
}
```

```
function fact(n, a) {
  while (true) {
    if (n === 0) {
      return a;
    }
    [n, a] = [n - 1, a * n];
    continue;
  }
}
```

and Fibonacci (remember a and b should be primed with 0 and 1):

```
function fib(n, a, b) {
  if (n === 0) {
    return a;
  }
  return fib(n - 1, b, a + b)
}
```

```
function fib(n, a, b) {
  while (true) {
    if (n === 0) {
      return a;
    }
    [n, a, b] = [n - 1, b, a + b];
    continue;
  }
}
```

Inline Expansion

Inlining saves the overhead of a function call at runtime. In practice it often exposes many other optimizations. Here's a trivial example to show the basic idea, where we **inline** the body of `g` into the point in `f` where it is called. (If this is the only call to `g`, or we inline all calls to `g`, then we don't have to generate a separate function for it at all):

```
function g(x, y) {
  let z = x * y;
  console.log(z / 5);
  return z + 1;
}
function f(x) {
  if (x > 0) {
    console.log(g(10, x));
  }
  return 1;
}
```

```
function f(x) {
  if (x > 0) {
    let __g_x = 10;
    let __g_y = x;
    let __g_z = __g_x * __g_y;
    console.log(__g_z / 5);
    let __g_returnValue = __g_z + 1;
    console.log(__g__returnValue);
  }
  return 1;
```

You should inline when the function is:

- Called only once in the program text
- Extremely small. Obvious candidates are those ridiculous getter methods like `int getX() { return x; }` in Java and C++.

You should strongly consider inlining when the function is:

- Frequently called at runtime (i.e., called in a loop)

You probably don't want to inline

- Recursive functions, but you can partially expand them (kind of like partial loop unrolling)

***Exercise**: Show how a recursive gcd function would be partially inlined. Do three "inlines."*

Further Reading

Nothing wrong with starting at the Wikipedia page for compiler optimizations and clicking from technique to technique and getting lost and absorbed in this incredibly cool subdiscipline of compilers.

Thinking about these topics is good. It might help you see computation in a new way.

Also, read whatever you can about Fran Allen, the 2006 Turing Award winner who did so much of the seminal work in optimizing compilers.

Summary

We've covered:

- ✅ Motivation
- ✅ Where optimizations happen in a compiler
- ✅ What we try to optimize
- ✅ A whole bunch of techniques
- ✅ Where to learn more